Multicore & GPU Programming : An Integrated Approach

Multicore Program Design

By G. Barlas

Objectives

- Learn the **PCAM** methodology for designing parallel programs.
- Use **task graphs** and **data dependency graphs** to identify parts of a computation that can execute in parallel.
- Learn popular **decomposition patterns** for breaking down the solution of a problem into parts that can execute concurrently.
- Learn major **program structure patterns** for writing parallel software, such as master-worker and fork/join.
- Understand the performance characteristics of decomposition patterns, such as pipelining.
- Learn how to combine a decomposition pattern with an appropriate program structure pattern.

The PCAM methodology

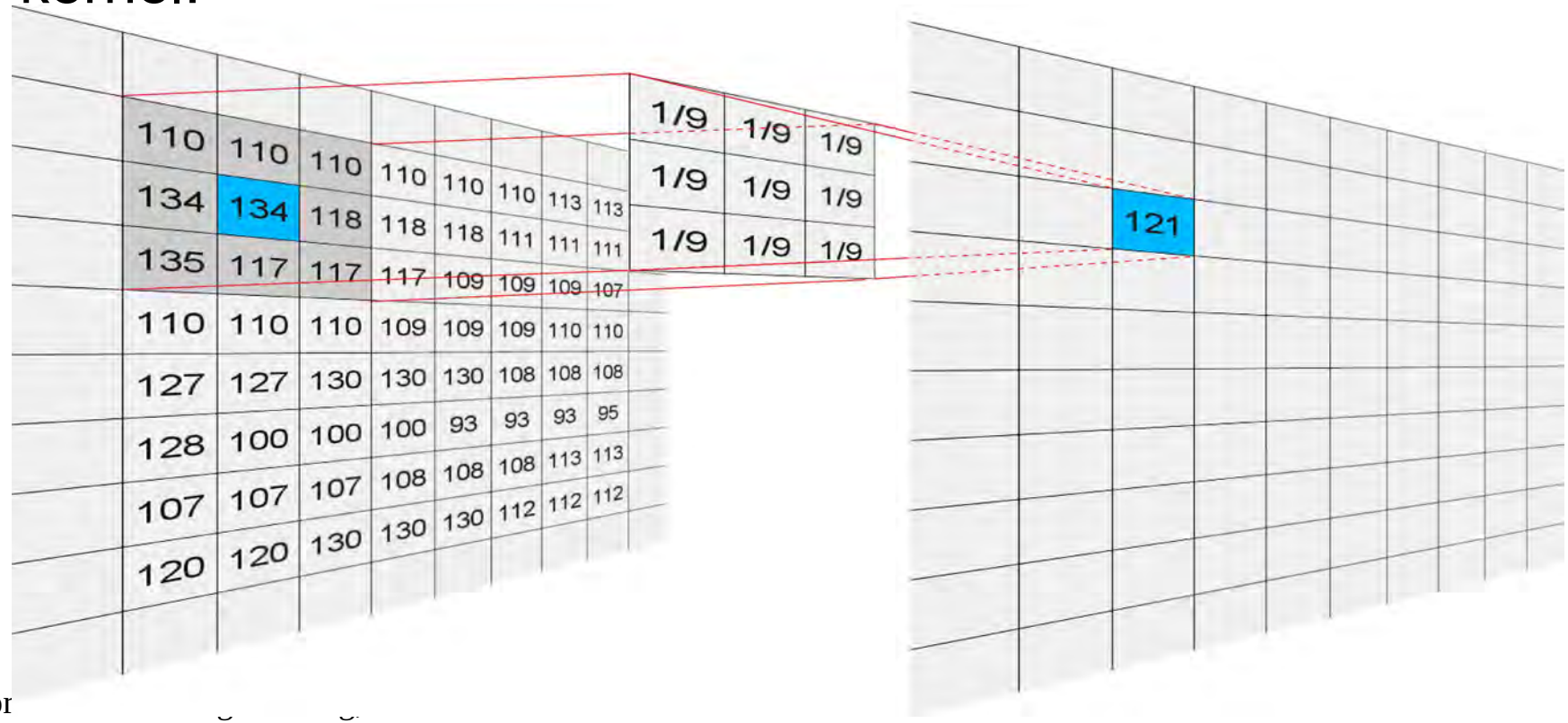
- PCAM stands for **P**artitioning, **C**ommunication, **A**gglomeration and **M**apping.
- Popularized by Ian Foster in his 1995 book, also available online at http://www.mcs.anl.gov/~itf/dbpp/text/book.html.
- **Partitioning** : The first step involves the break-up of the computation in as many individual pieces as possible.
  - The break-up can be function-oriented,(called **functional decomposition**), or data-oriented, (called **domain** or **data decomposition**).
- **Communication** : identify the volume of data „flowing“ between the tasks generated in the previous step.
  - The combination of the first two steps results in the creation of the **task dependency graph**, with nodes representing tasks and edges representing communication volume.

The PCAM methodology, cont.

- **Agglomeration** : Communication is hampering parallel computation. One way to eliminate it, is to group together tasks.
  - The number of groups produced at this stage should be as a rule-of-thumb, one order of magnitude bigger than the number of compute nodes available.
- **Mapping** : pair groups of tasks and nodes that will execute them. Objectives:
  - (a) load balance the nodes
  - (b) reduce communication overhead
- Partitioning and Communication are algorithm specific.
- Agglomeration and Mapping are platform specific.

PCAM example

- Image convolution, which can be used for noise filtering, edge detection, or other applications, based on the kernel used.
- The kernel is a square matrix with weights that are used in the calculation of the new pixel data. Example for a blur effect kernel:



PCAM example, cont.

- Convolution between a kernel *K* of -odd- size *n*, and an image *f* is defined by the formula:

$$g(x,y) = \sum_{i=-n2}^{n2} \sum_{j=-n2}^{n2} k(n2+i, n2+j) f(x-i, y-j)$$

where $n2 = \lfloor \frac{n}{2} \rfloor$

- If for example, a 3x3 kernel is used :

$$K = \begin{vmatrix} k_{0,0} & k_{0,1} & k_{0,2} \\ k_{1,0} & k_{1,1} & k_{1,2} \\ k_{2,0} & k_{2,1} & k_{2,2} \end{vmatrix}$$

PCAM example, cont.

- Then for each pixel at row *i* and column *j*, the new pixel value $v'_{i,j}$ is produced according to the formula:

$$v'_{i,j} = v_{i-1,j-1} \cdot k_{2,2} + v_{i-1,j} \cdot k_{2,1} + v_{i-1,j+1} \cdot k_{2,0} +$$
$$v_{i,j-1} \cdot k_{1,2} + v_{i,j} \cdot k_{1,1} + v_{i,j+1} \cdot k_{1,0} +$$
$$v_{i+1,j-1} k_{0,2} \cdot + v_{i+1,j} \cdot k_{0,1} + v_{i+1,j+1} \cdot k_{0,0}$$

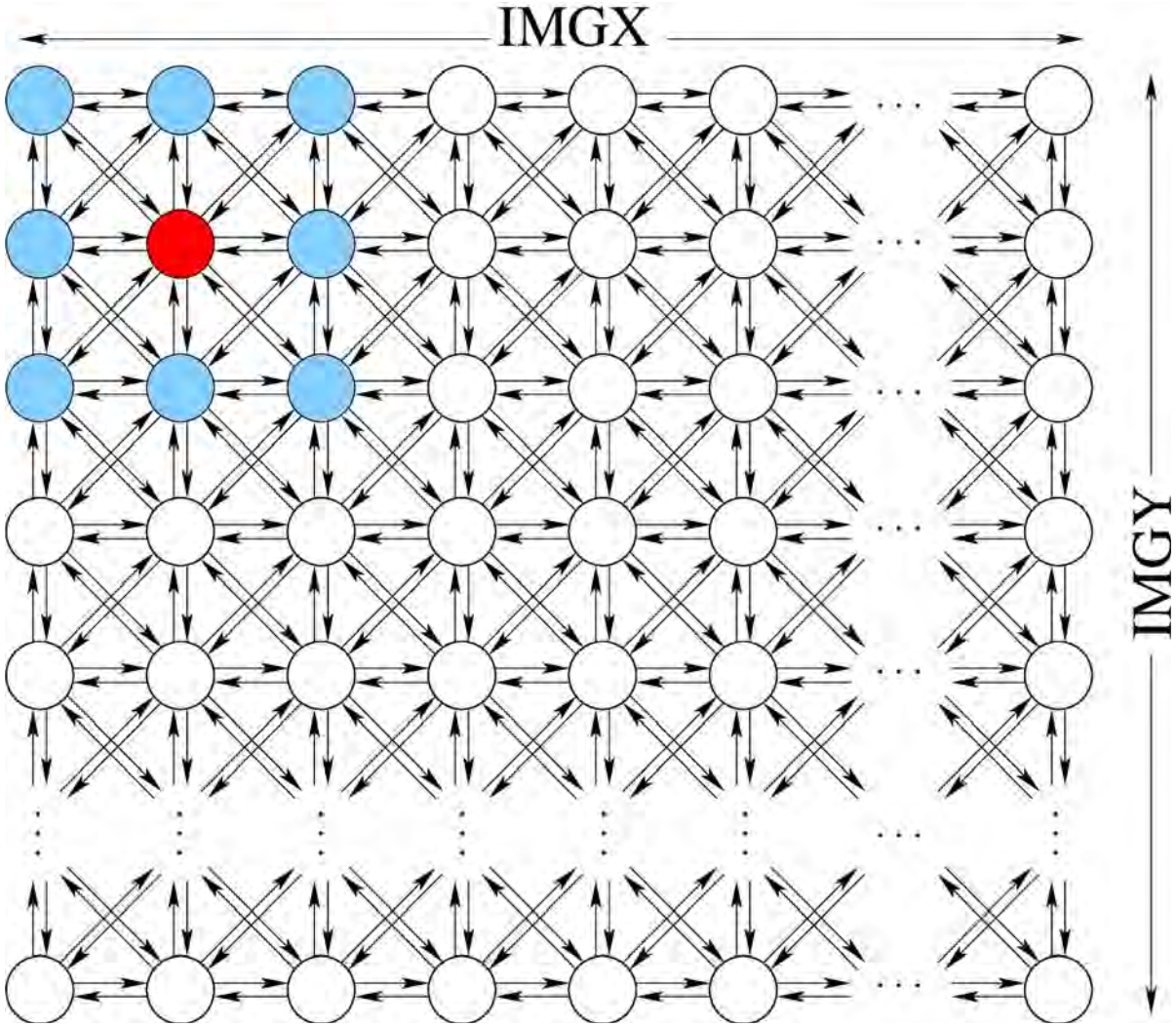
PCAM example, cont.

- Pseudocode of sequential implementation:

```
int img[IMGY+2][IMGX+2];
int filt[IMGY][IMGX];
int n2 = n/2;
for(int x=1;x <= IMGX; x++) {
    for(int y=1; y <= IMGY ; y++)  {
        int newV=0;
        for(int i= -n2; i<= n2; i++)
            for(int j= -n2; j<= n2; j++)
                newV += img[ y - j][ x - i ] * k[n2 + j][n2 + i];
        filt[y-1][x-1] = newV;
        }
    }
```

- What happens if we unroll the two outer loops?

PCAM example : partitioning



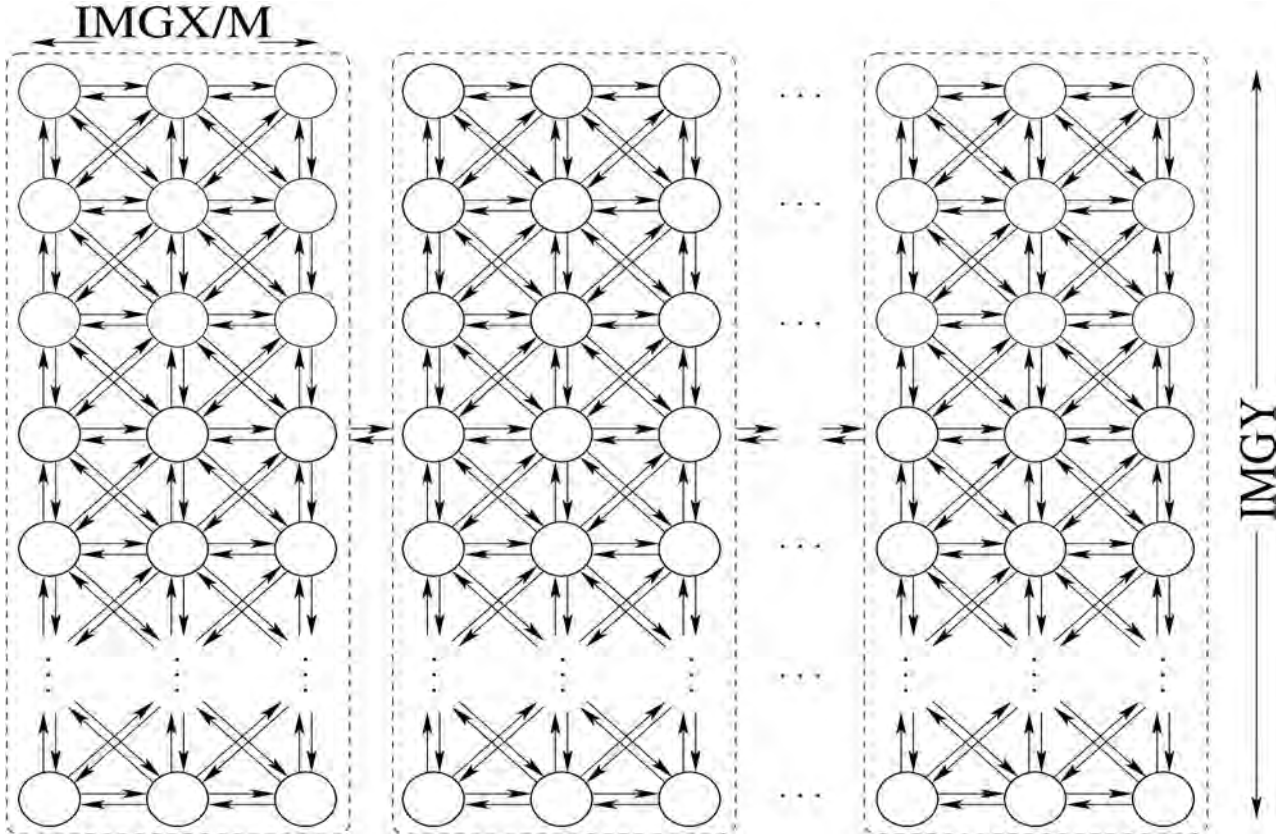
PCAM example : communication

- Total volume of data that need to be communicated:

$$totalComm = 8 \cdot IMGX \cdot IMGY +$$
$$-3 \cdot 2 \cdot (IMGX - 2) - 3 \cdot 2 \cdot (IMGY - 2) - 4 \cdot 5 =$$
$$8 \cdot IMGX \cdot IMGY - 6 \cdot (IMGX + IMGY) + 4$$

PCAM example : agglomeration

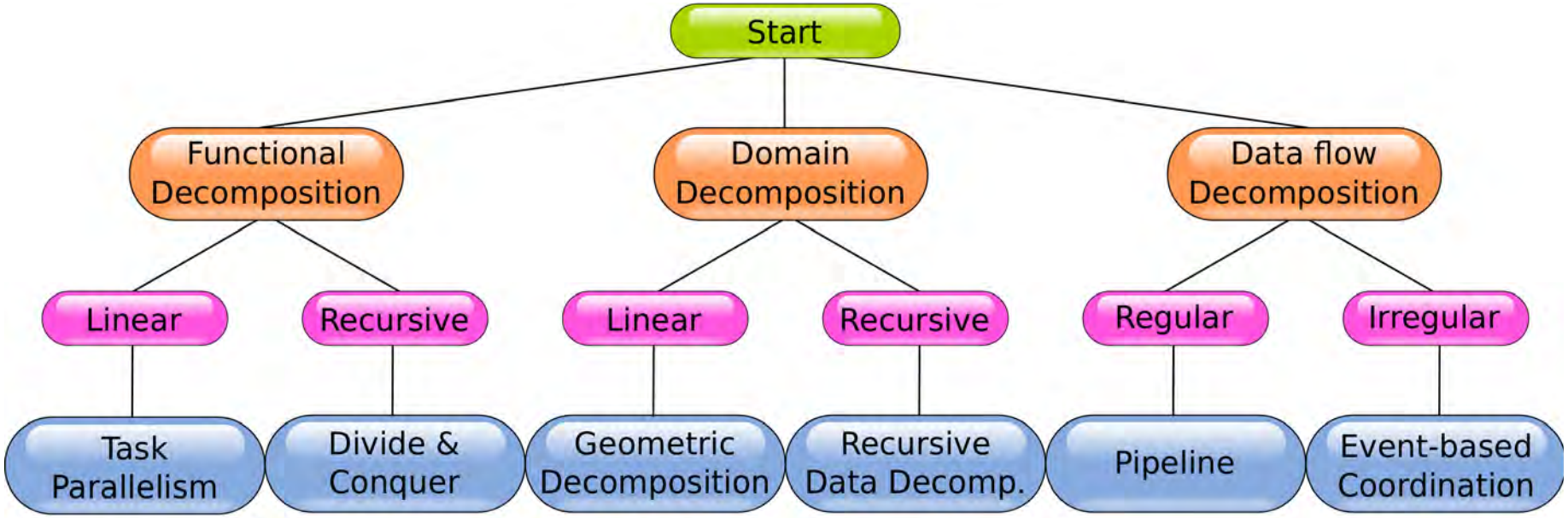
- Tasks can be grouped together in 1D or 2D:



- Total volume communicated now?

Decomposition patterns

- PCAM is generic but a lot of work.
- Faster solution provided by decomposition patterns:



Task parallelism

- Break application into tasks, decided offline (a priori).
- Generally this scheme does not have *strong scalability.*
- Example, game loop:

```
while(true) {
    readUserInput();
    drawScreen();
    playSounds();
    strategize();
    }
```

Task parallelism (2)

- Turn each function into a task.
- What are the barriers for?

```
Task1 {
   while(true) {
      readUserInput();
      barrier();
      }
Task2 {
   while(true) {
      drawScreen();
      barrier();
      }
```

```
Task3 {
   while(true) {
      playSounds();
      barrier();
      }
Task4 {
   while(true) {
      strategize();
      barrier();
   }
```

Divide-n-conquer

- Popular algorithmic technique:

```
// Input: A
DnD(A) {
   if(A is a base case)
      return solution(A);
   else {
      split A into N subproblems B[N];
      for(int i=0;i<N;i++)
         sol[i] = DnD( B[i] );        //
      return mergeSolution(sol);
   }
}
```

Divide-n-conquer (2)

- Generic parallel divide-n-conquer:

```
// Input: A
DnD(A) {
  if(isBaseCase( A ))
     return solution(A);
  else {
     if( bigEnoughForSplit( A ) ) { // if problem is big enough
        split A into N subproblems B[N];
        for(int i=0;i<N;i++)
           task[i] = newTask( DnD( B[i] ) );    // non-blocking

        for(int i=0;i<N;i++)
           sol[i] = getTaskResult( task[i] );  // blocking results ↩
                wait

        return mergeSolution( sol );
     }
     else {                              // else solve sequentially
        return solution( A );
     }
  }
}
```

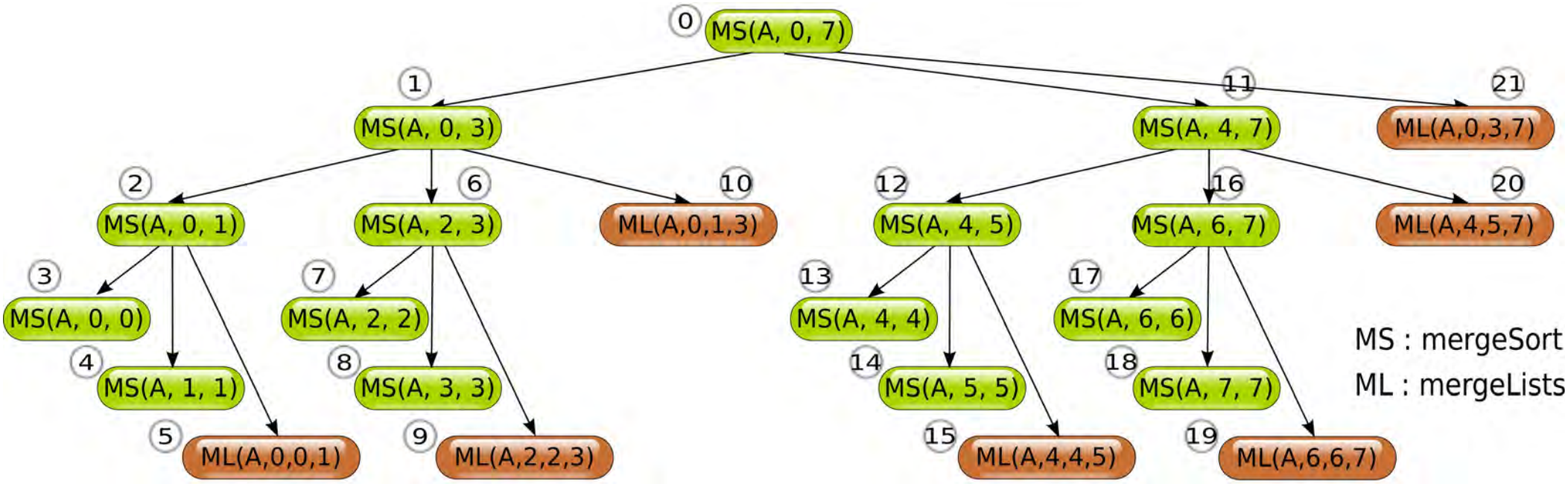
Divide-n-conquer example

- Mergesort:

```
// Input: array A and endpoints st, end
mergeSort(A, st, end) {
   if(end > st) {
      middle = (st+end) / 2;
      mergeSort(A, st, middle);
      mergeSort(A, middle+1, end);
      mergeLists(A, st, middle, end);
   }
}
```

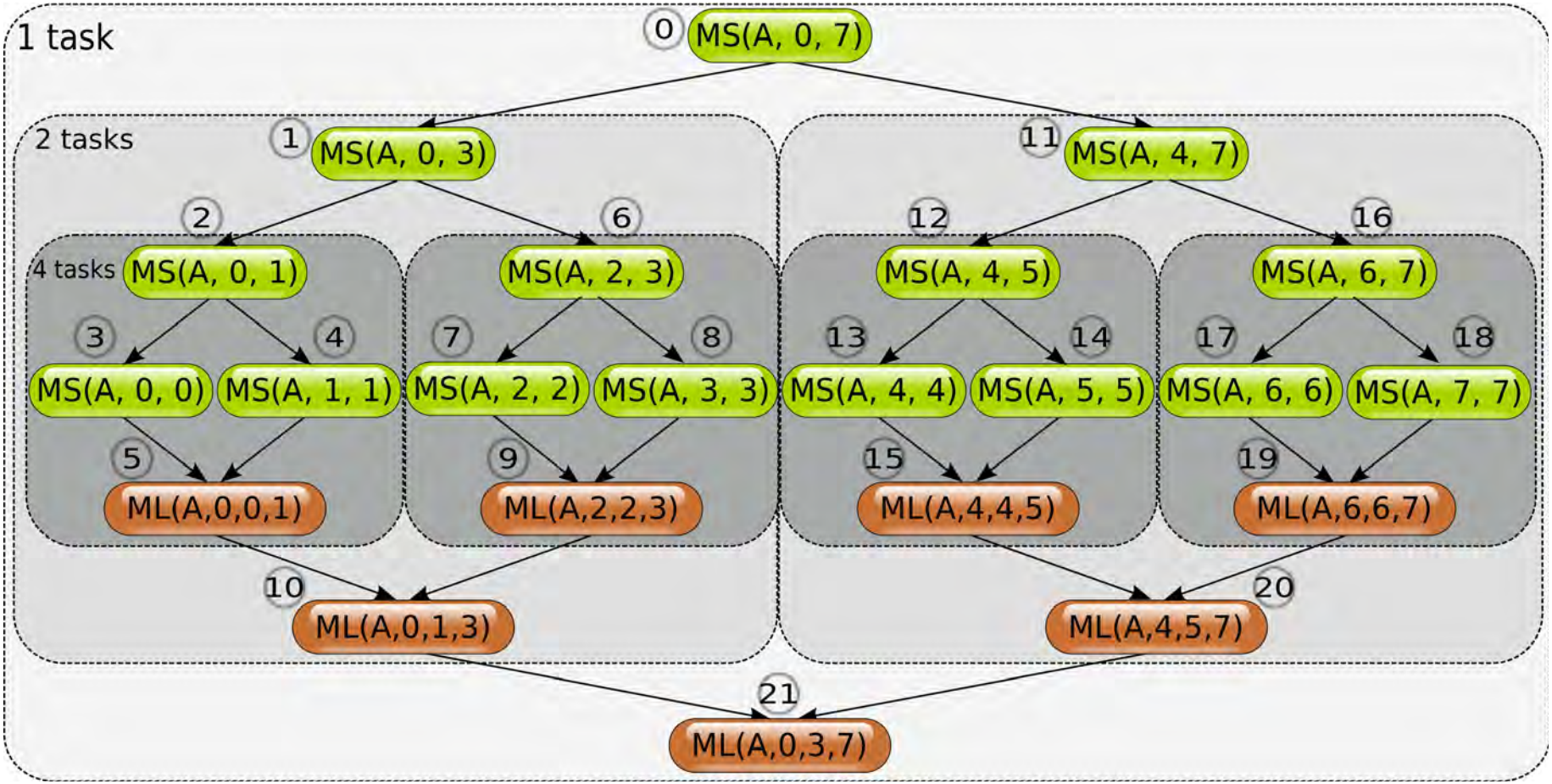
Divide-n-conquer example (2)

- Mergesort call graph:



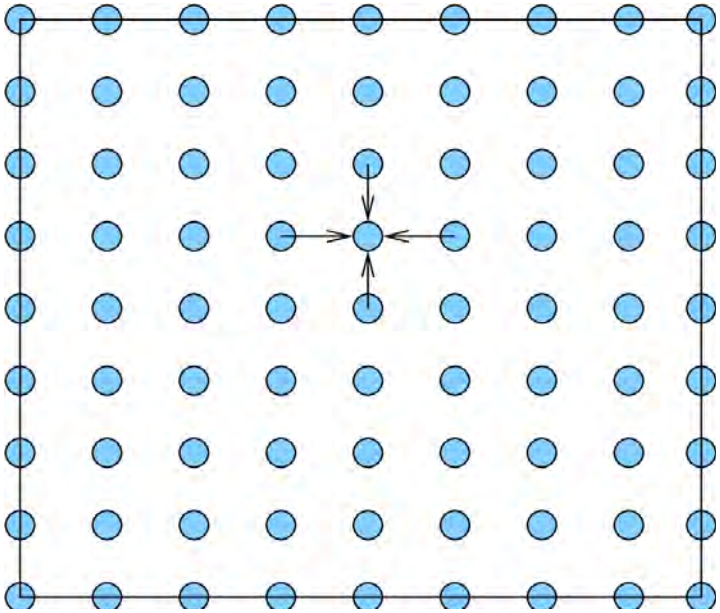
Divide-n-conquer example (3)

- Mergesort dependency graph:



Geometric decomposition

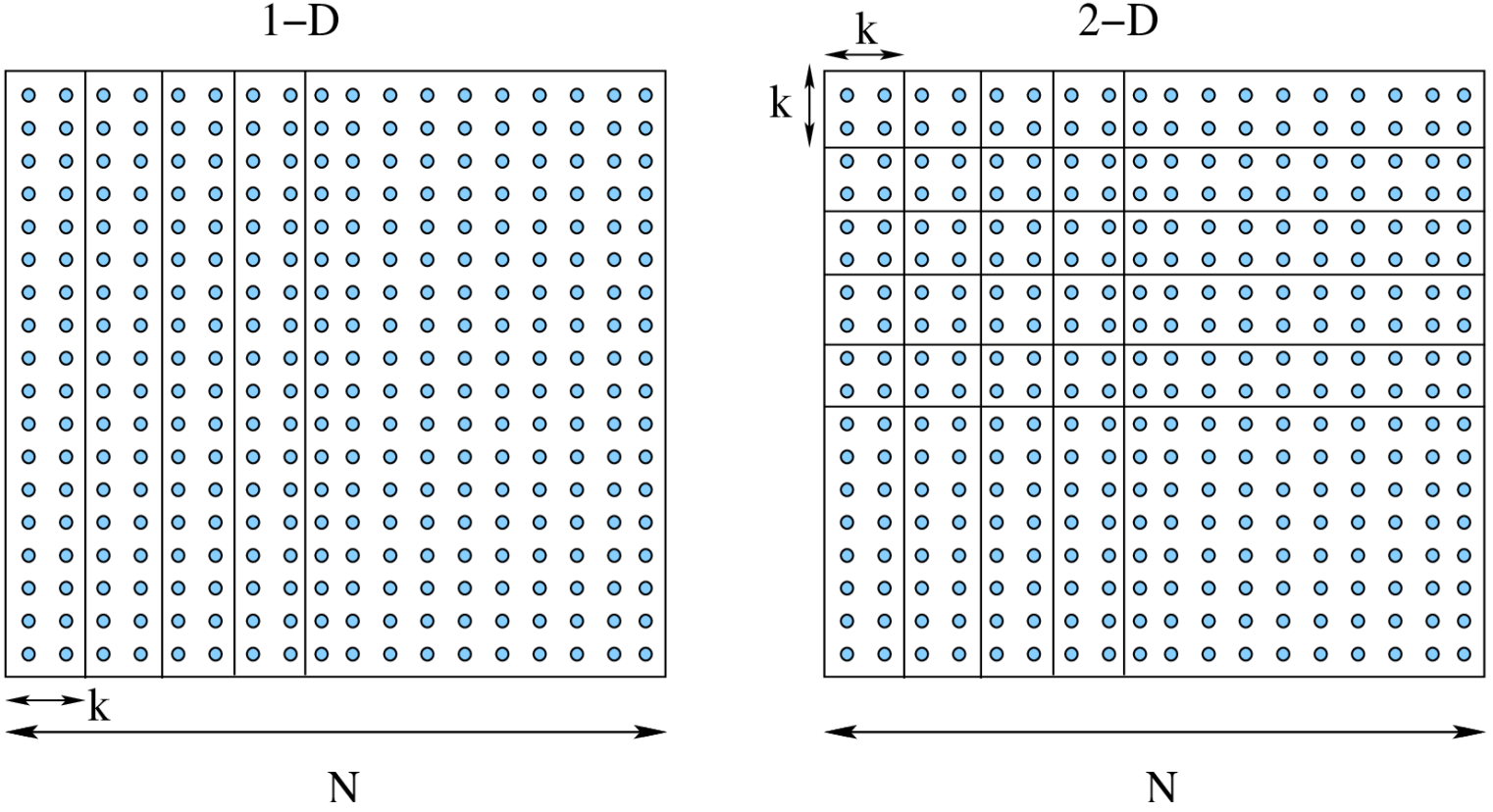
- If the problem's data are organized in linear structures such as arrays, 2D matrices, etc. then the data can be split along one or more dimensions and assigned to different tasks.
- Example: simulating heat diffusion on a 2D surface.



$$u'[i,j] = u[i,j] + \delta t \cdot \frac{u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1] - 4u[i,j]}{h^2}$$

Geometric decomposition (2)

- 1-D and 2-D decomposition is possible:



- Which one is preferable?

Geometric decomposition (3)

- Assuming that:
  - N : the number of cells per dimension
  - k : number of cells per division along the dimension of decomposition
  - P : the number of compute nodes
  - t_{comp} : processing cost per single update, for a single cell
  - t_{comm} : communication cost per data item send between two compute nodes
  - t_{start} : communication start-up latency
  - Full-duplex communication links

Geometric decomposition (4)

- Then for 1-D decomposition we have:

$$comp_{1D} = k \cdot N \cdot t_{comp} = \frac{N^2}{P} \cdot t_{comp}$$

$$comm_{1D} = 2 \cdot (t_{start} + t_{comm} N)$$

- And for 2-D we have:

$$comp_{2D} = k^2 t_{comp} = \frac{N^2}{P} t_{comp}$$

$$comm_{2D} = 4\left(t_{start} + t_{comm} \cdot k\right) = 4\left(t_{start} + t_{comm} \cdot \frac{N}{\sqrt{P}}\right)$$

Geometric decomposition (5)

- 1-D is better than 2-D if:

$$comm_{1D} + comp_{1D} < comm_{2D} + comp_{2D} \Rightarrow$$

$$t_{start} > t_{comm} N \left(1 - \frac{2}{\sqrt{P}}\right)$$

Recursive data decomposition

- Recursive data structures such as trees, lists, or graphs, cannot be partitioned in the straightforward manner of the previous section.
- In recursive data decomposition the data structure is decomposed into individual elements. Each element (or groups of elements) is then assigned to a separate compute node.
- The process may involve a **modification of the original algorithm** so that concurrent operations can take place.
- The way of operation mimics **dynamic programming**, where bigger problems are solved based on the stored solutions of smaller problems.

Recursive data decomp. example

- The **prefix-sum** problem: calculating the partial sum of all the elements in an array that precede a location:

$$S_i = \sum_{j=0}^{i} A_j$$

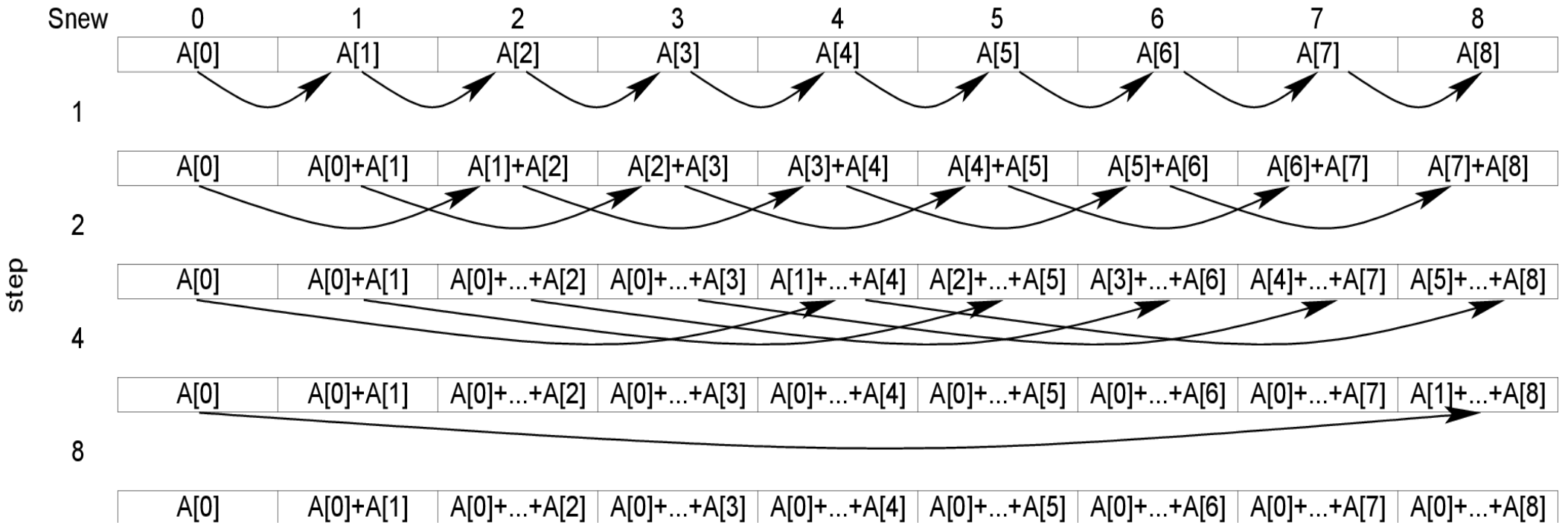
- Sequential implementation:

```
S[0] = A[0];
for(int i=1; i< N; i++)
    S[i] = S[i-1] + A[i];
```

Prefix-sum : alternative sequential

```
int S[2][N];

S[1][0] = A[0];
S[0][0] = A[0];

int *Snew, *Sold, *aux;
Snew = S[0];
Sold = A;
aux = S[1];

int step=1;
while(step < N) {
  int pos=0;

  while(pos + step < N) {
     Snew[pos + step] = Sold[ pos ] + Sold[pos + step];
     pos ++;
    }

  Sold = Snew;
  Snew = aux;
  aux = Sold;
  step *= 2;
 }
```

Prefix-sum : alternative visualization



Prefix-sum : alternative implementation analysis

- The time complexity of the original implementation is:

$$\Theta(N)$$

- The time complexity of the alternative is:

$$\Theta(NlgN)$$

- BUT, we can start execution on *N* (actually *N-1* are enough for the first step) CPUs, and every step can use $N - step$ CPUs!

- Overall execution would require $\lceil lgN \rceil$ steps!

Prefix-sum : alternative implementation analysis

- So if *N-1* CPUs were available, we could achieve:

$$speedup = \frac{N-1}{\lceil lgN \rceil}$$

- The efficiency would not be that great:

$$efficiency = \frac{speedup}{N-1} = \frac{1}{\lceil lgN \rceil}$$

- In practice, we do not have that many nodes, so operations on individual elements have to be grouped together.

Prefix-sum : parallel pseudocode

```
int step=1;
while(step < N) {

  parallel do {
     int pos=ID;

     while(pos + step < N) {
        Snew[pos + step] = Sold[ pos ] + Sold[pos + step];
        pos += P;
     }
  }
  barrier();

  Sold = Snew;
  Snew = aux;
  aux = Sold;
  step *= 2;
}
```

Each node must have its own unique ID

Advance by the number of CPUs

Wait for all updates to finish

Prefix-sum : parallel alg. analysis

$$t_{par} = t_c \left(\lceil \frac{N-1}{P} \rceil + \lceil \frac{N-2}{P} \rceil + \cdots + \lceil \frac{N-2^{lgN-1}}{P} \rceil \right) =$$

$$= t_c \sum_{i=0}^{lgN-1} \lceil \frac{N-2^i}{P} \rceil \approx t_c \sum_{i=0}^{lgN-1} \frac{N-2^i}{P} =$$

$$= t_c \left(\frac{N}{P} \sum_{i=0}^{lgN-1} 1 - \frac{1}{P} \sum_{i=0}^{lgN-1} 2^i \right) = t_c \left(\frac{N}{P} lgN - \frac{1}{P} \left(2^{lgN} - 1 \right) \right) =$$

$$= t_c \left(\frac{N}{P} lgN - \frac{N}{P} + \frac{1}{P} \right) = t_c \left(\frac{N}{P} \left(lgN - 1 \right) + \frac{1}{P} \right)$$

Prefix-sum : parallel alg. analysis (2)

- In order to obtain a speedup greater than one:

$$speedup = \frac{t_{seq}}{t_{par}} = \frac{t_c\,(N-1)}{t_c\left(\frac{N}{P}\,(lgN-1)+\frac{1}{P}\right)} > 1 \Rightarrow$$

$$(N-1) > \frac{N}{P}\,(lgN-1)+\frac{1}{P} \Rightarrow (\text{multipling both sides by } \frac{P}{N-1})$$

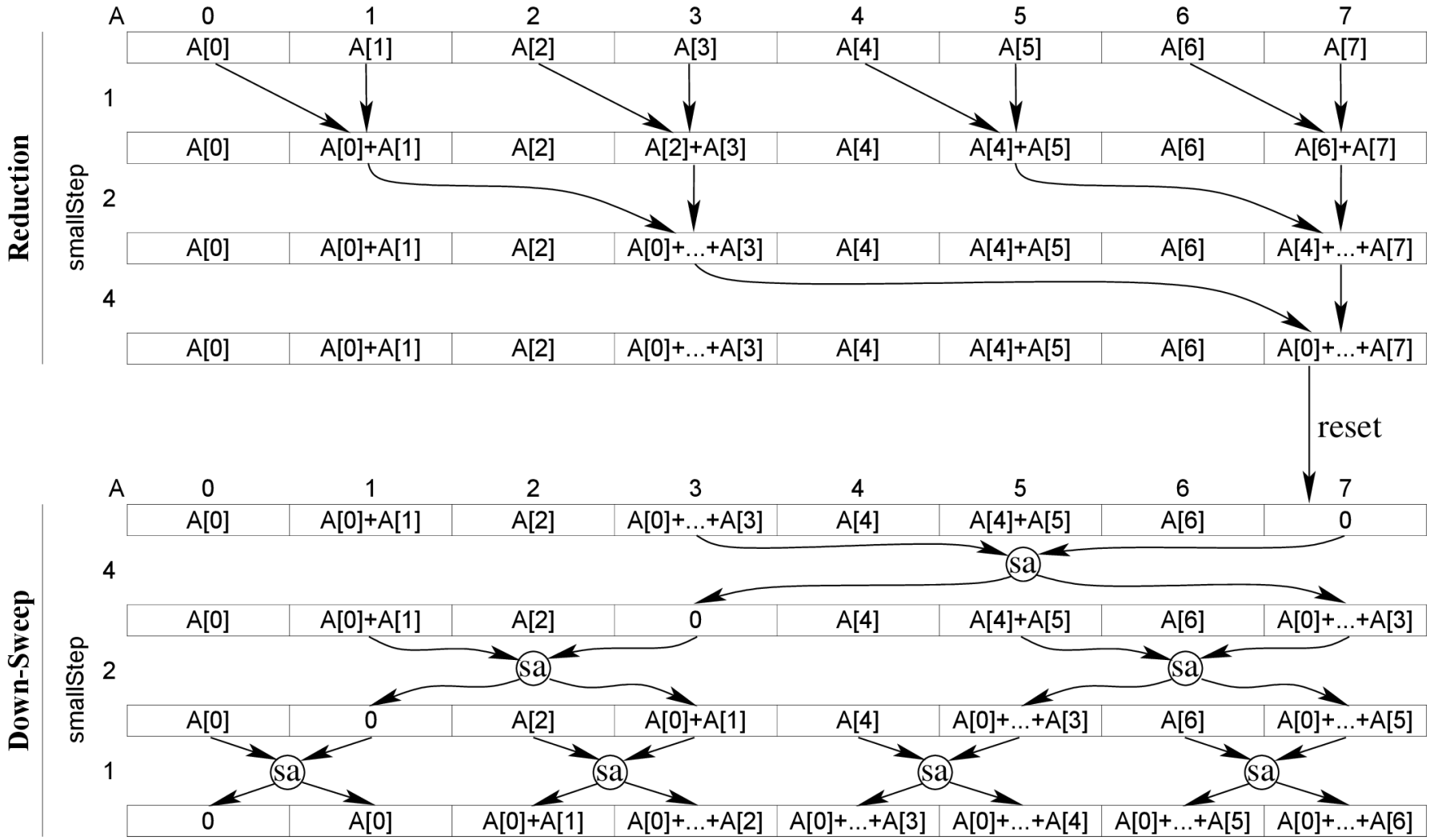
$$P > \frac{N}{N-1}(lgN-1)+\frac{1}{N-1}$$

- So, for $N=2^{20}$ we need more than 19 CPUs.

Blelloch's prefix-sum algorithm

- Blelloch proposed a solution comprising of two phases:
  - A reduction phase, and a
  - Down-sweep phase
- Blelloch’s algorithm works by treating the array as the leaf level of an implicit perfect binary tree. During the reduction phase, we sum up the children of each internal node and store the result in their parent.
- At the end of the reduction phase, the “root” contains the total sum of all the data.
- During the down-sweep phase, we replace the root with zero, and we traverse the ``tree'' in reverse, starting from the root and descending to the leaves. At each level, we switch the values of left and right children of an internal node, and deposit their sum in the right child.
- All the calculations are in-place.

Blelloch's prefix-sum algorithm example



sa : swap and add

Blelloch's prefix-sum algorithm

```
4   // reduction phase
5   int step = 2;
6   int smallStep = 1;
7   while (smallStep < N)
8     {
9       // assume that the for is partitioned between the nodes
10      parallel for (int i = 0; i < N; i += step)
11          d[i + step - 1] += d[i + smallStep - 1];
12
13      smallStep = step;
14      step *= 2;
15    }
16
17  // down-sweep phase
18  d[N - 1] = 0;
19  step = smallStep;
20  smallStep /= 2;
21  while (smallStep > 0)
22    {
23      parallel for (int i = 0; i < N; i += step)
24        {
25          int temp = d[i + smallStep - 1];
26          d[i + smallStep - 1] = d[i + step - 1];
27          d[i + step - 1] += temp;
28        }
29
30      step = smallStep;
31      smallStep /= 2;
32    }
```

Blelloch's algorithm analysis

- Assuming t_c is the cost of a single addition, or a single swap, then given 2^{M-1} processors, with $M=\lceil \log_2(N) \rceil$, we can complete the calculation in:

$$t_{par}^{(Blel)} = t_c \cdot M + t_c \cdot 2 \cdot M = 3 \cdot t_c \cdot M$$

- Of course, this is an extreme scenario, as we typically do not have this kind of computing resources available.
- If P processors were at our disposal, then the execution time could be approximated by:

$$t_{par}^{(Blel)} = 3 \cdot t_c \left(\lceil \frac{N/2}{P} \rceil + \lceil \frac{N/4}{P} \rceil + \cdots + \lceil \frac{1}{P} \rceil \right) =$$

$$= \frac{3 \cdot t_c}{P} (2^{lg(N)} - 1) \approx \frac{3 \cdot t_c}{P} N$$

- This is a lower bound as the sum in the rhs is underestimated by a $\log_2(P))$ term.

Pipeline decomposition

- A pipeline is the software/hardware equivalent of an assembly line.
- An item can pass through a pipeline made-up of several **stages**, each stage applying a particular operation on it.
- Examples:
  - **CPU architectures** : machine instructions in modern CPUs are executed in stages that form a pipeline.
  - **Signal Processing** : many signal processing algorithms are formulated as pipelines. (e.g. filtering with FFT).
  - **Graphics rendering**
  - **Shell programming** : *nix provides the capability to feed the console output of one command at the console input of another, thus forming a command pipeline.

Pipeline decomposition, cont.

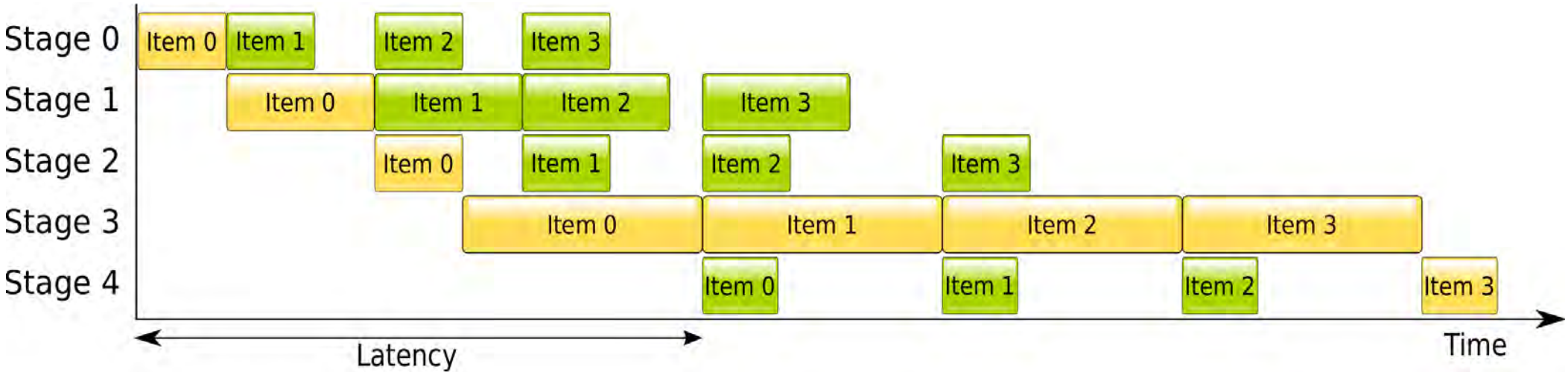
- A pipeline stage performs the following:

```
initialize();
while( moreData ) {
    readDataFromPreviousStage();
    process();
    sendDataToNextStage();
}
```

- Communications are usually synchronous : sending and receiving happens at the same time.

Pipeline Gantt chart

- Example of a pipeline. Communication costs are assumed to be zero.
- Overall time is dominated by slowest stage.



Pipeline analysis

- If S_l is the slowest stage in a *M*-stage pipeline:

$$t_{total} = \sum_{j=0}^{l-1} t_j + N \cdot t_l + \sum_{j=l+1}^{M-1} t_j$$

- Rate of computation:

$$rate = \frac{N}{\sum_{j=0}^{l-1} t_j + N \cdot t_l + \sum_{j=l+1}^{M-1} t_j}$$

- Latency:

$$latency = \sum_{j=0}^{M-2} t_j$$

Pipeline analysis for uniform stages

- Total time

$$t_{total} = \sum_{j=0}^{M-2} t + N \cdot t = (N + M - 1)t$$

- Rate of computation:

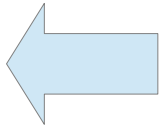
$$rate = \frac{N}{(N + M - 1)t}$$

- Latency:

$$latency = (M - 1)t$$

- Speedup

$$speedup = \frac{N \cdot M \cdot t}{(N + M - 1)t} \approx M$$

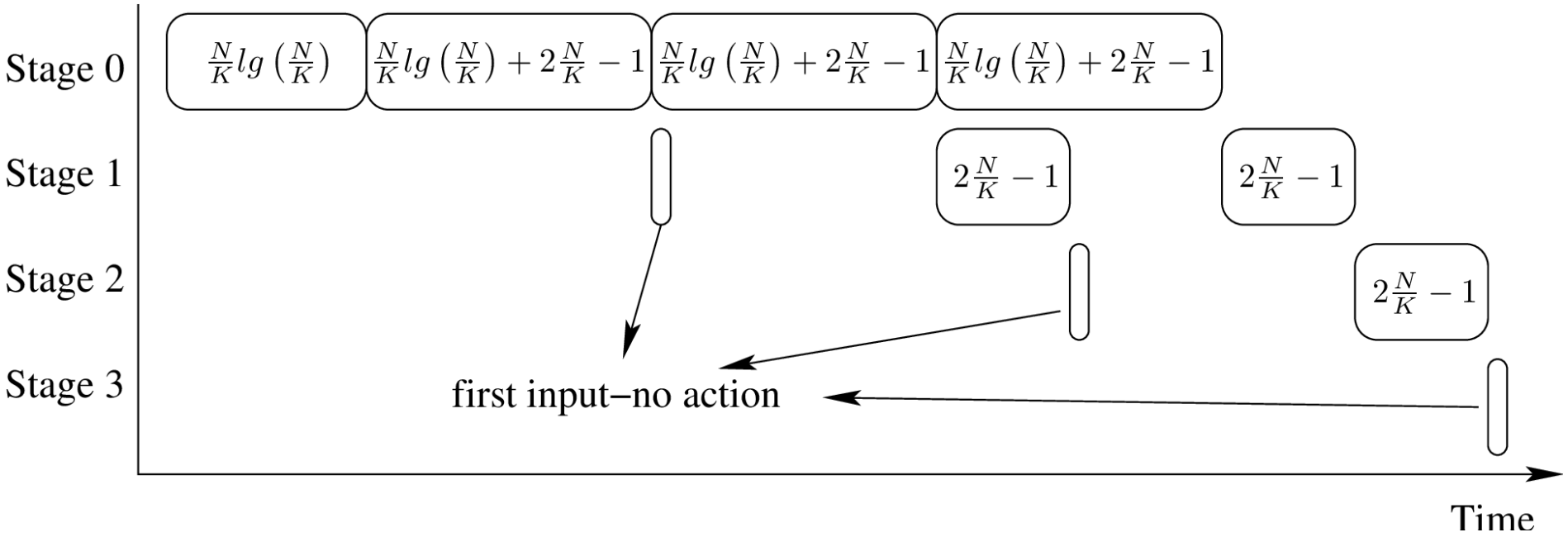


Pipeline sort, version 2

- Using *K* stages to sort *N* items, communicated in batches on *N/K* items:

```
void stage(int sID)                          // sID is the stage ID
  T part1[] = readFromStage(sID-1); // reads an array of data
  if(sID == 0) MergeSort(part1);    // a typical sequential sort
  for(int j=0; j< K - sID -1 ; j++ ) {
     T part2[] = readFromStage(sID-1);
     if(sID == 0) MergeSort(part2);
     MergeLists(part1, part2);      // merge part2 into part1
     sendToStage(sID+1, secondHalf(part1));
  }
  store(result , part1, sID);       // deposit partial result
}
```

Pipeline sort, version 2 illustration



Pipeline sort, version 2 analysis

- Total execution time, assuming no communication cost:

$$K\frac{N}{K}lg\left(\frac{N}{K}\right)+(K-1)\left(2\frac{N}{K}-1\right)+(K-2)\left(2\frac{N}{K}-1\right)=$$

$$Nlg\left(\frac{N}{K}\right)+(2K-3)\left(2\frac{N}{K}-1\right)\approx N\cdot lg\left(\frac{N}{K}\right)$$

- Speedup upper bound by:

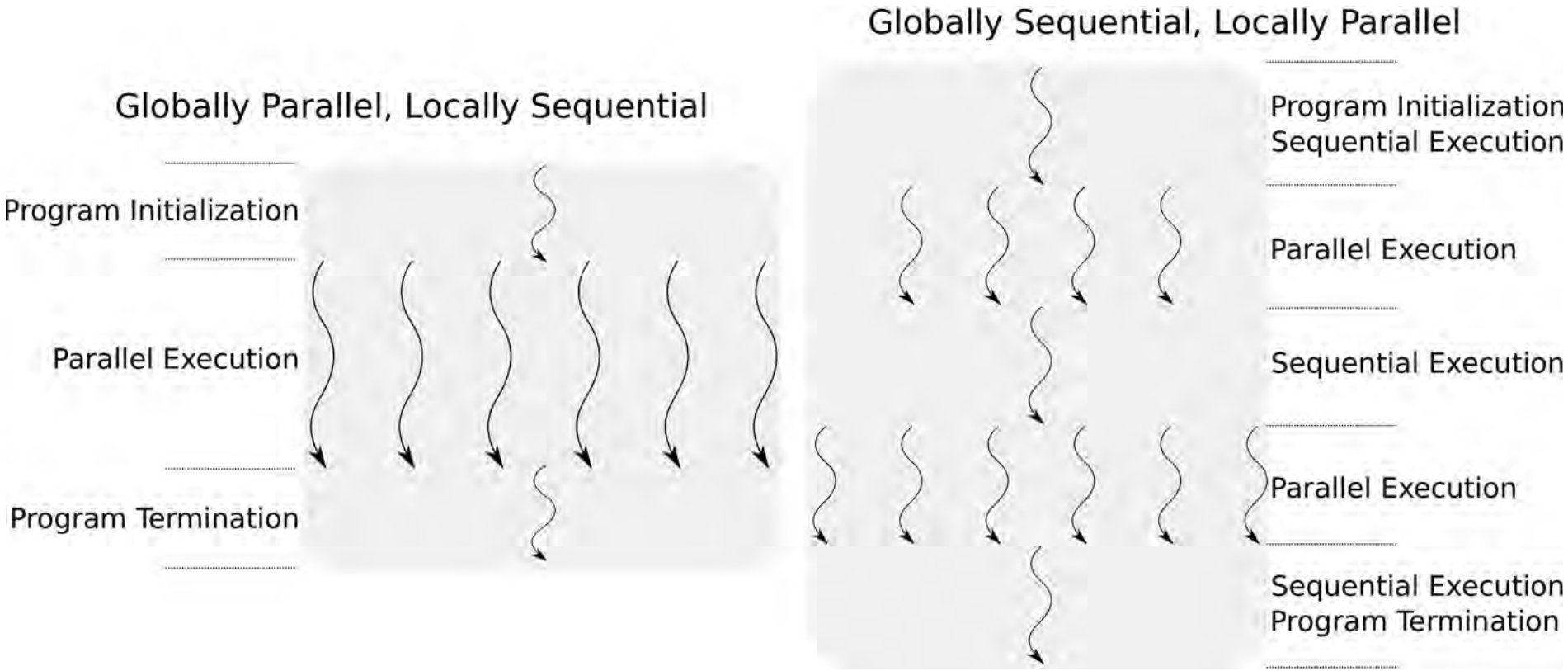
$$speedup_{max}=\frac{Nlg(N)}{Nlg\left(\frac{N}{K}\right)}=log_{\frac{N}{K}}(N)$$

Program structure patterns

We can distinguish the parallel program structure patterns into two major categories:

- **Globally Parallel, Locally Sequential** (GPLS) : this means that the application is able to perform multiple tasks concurrently, with each task running sequentially. Patterns that fall in this category include:
  - Single-Program, Multiple Data
  - Multiple-Program, Multiple Data
  - Master-Worker
  - Map-reduce
- **Globally Sequential, Locally Parallel** (GSLP) : this means that the application executes as a sequential program, with individual parts of it running in parallel when requested. Patterns that fall in this category include:
  - Fork/join
  - Loop parallelism

Program structure patterns, cont.



Single program multiple data

- Keeps all the application logic in a single program.
- Typical program structure involves:
  - **Program initialization** : e.g. runtime system initialization
  - **Obtaining a unique identifier** : identifiers are numbered from 0, enumerating the threads or processes used. Some systems use vector identifiers (e.g. CUDA, OpenCL).
  - **Running the program** : execution path diversified based on ID.
  - **Shutting-down the program** : clean-up, saving results, etc.

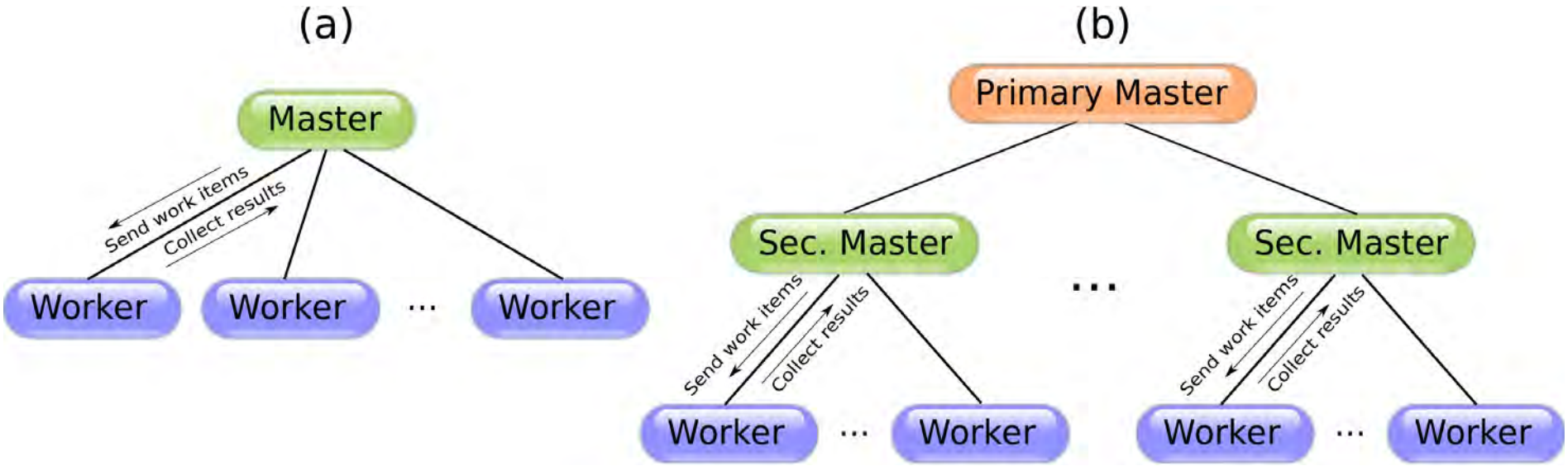
Multiple program multiple data

- SPMD fails when:
  - Memory requirements are too high for all nodes.
  - Heterogeneous platforms are involved.
- Execution steps are identical as SPMD.
- But deployment involves different programs.
- Both SPMD and MPMD are supported by MPI.

Master-worker

- Two kinds of components.
- Master (one or more) is responsible for:
  - Handing out pieces of work to workers.
  - Collecting the results of the computations from the workers.
  - Performing I/O duties on behalf of the workers, i.e. sending them the data that they are supposed to process, or accessing a file.
  - Interacting with the user.
- Good for implicit load balancing.
- Master can be a bottleneck.

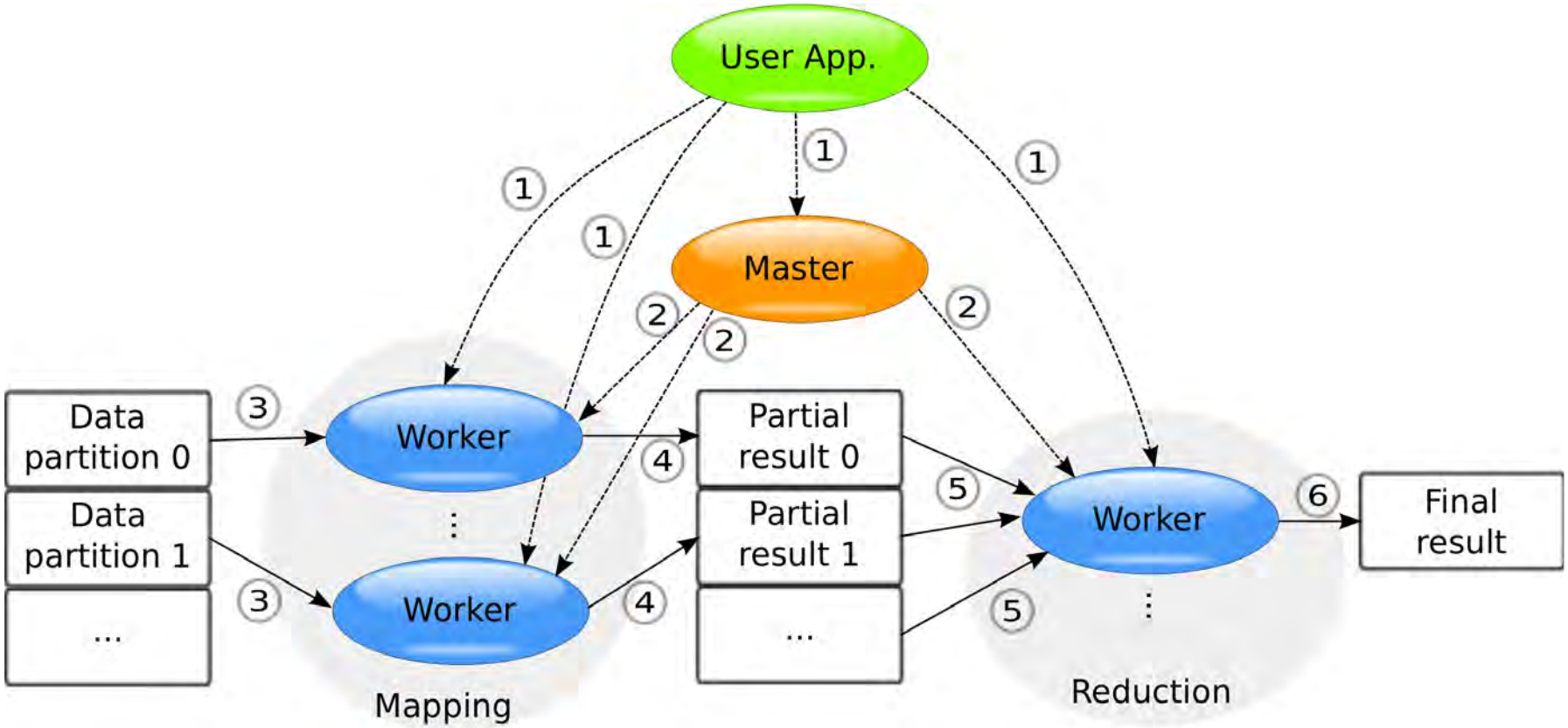
Master-worker set-ups



Map-reduce

- Variation of master-worker pattern.
- Made popular by Google's search engine implementation.
- Master coordinates the whole operation.
- Workers run two types of tasks:
  - **Map** : apply a function on data, resulting in a set of partial results
  - **Reduce** : collect the partial results and derive the complete one.
- Map and reduce workers can vary in number.

Map-reduce, cont.



Fork/join

- Single, parent thread of execution (GSLP).
- Dynamic creation of children tasks at run-time.
- Tasks may run via spawning of threads, or via use of a static pool of threads.
- Children tasks have to finish for parent thread to continue.
- OpenMP uses this pattern.

Fork/join example

- Parallel quicksort pseudocode:

```
template <typename T>
void QuickSort<T>(T [] inp, int N) {
  if(N <= THRES)  // for small input sort sequentially
      sequentialQuickSort(inp, N);
  else {
     int pos = PartitionData(inp, N);        // split data into two parts
     Task t = new Task( QuickSort(inp, pos)); // new task for left part
     t.run();                                  // non-blocking call
     QuickSort(inp + pos+ 1, N-pos-1);     // keep sorting the second part
     waitUntilDone(t);                     // wait for child to finish
  }
}
```

Quicksort analysis

- The number of **extra** tasks *T(N)* is :

$$T(N) = \begin{cases} 0 & \text{if } N \leq THRES \\ 1 + 2T(\frac{N-1}{2}) \approx 1 + 2T(\frac{N}{2}) & \text{if } N > THRES \end{cases}$$

- Backward-substitution yields the answer:

$$T(N) = \frac{N}{THRES} - 1$$

- So, for 2^{20} items and THRES=2^{10}, we need 1023 tasks.

Loop parallelism

- Employed for migration of legacy/sequential software to multicore.
- Focuses on breaking up loops by manipulating the loop control variable.
- A loop has to be in a particular form to support this.
- Limited scalability, but limited development effort as well.
- Supported by OpenMP.

Decomposition – program structure

Program Structure Patterns	Decomposition Patterns: Task Parallelism	Divide and Conquer	Geometric Data	Recursive Data	Pipeline	Event-based Coordination
Single-Program Multiple Data	√	√	√	√	√	√
Multiple-Program Multiple Data	√	√	√	√	√	√
Master-Worker	√	√	√	√		
Map-Reduce		√	√	√		
Fork/Join	√	√	√	√	√	√
Loop Parallelism		√	√			